

Regex To DFA Project - Step By Step Code Explanation

1. PROJECT GOAL

This project converts a regular expression into a working minimized DFA. It is designed as a university Formal Languages and Automata Theory project, so the code is separated into small classes and the console menu is easy to present.

The full path is: user input regex -> validation and postfix conversion -> Thompson NFA -> subset DFA -> minimized DFA -> string testing.

2. SUPPORTED REGEX RULES

The user may enter normal symbols such as a, b, 0, and 1. The operator | means union, * means zero or more, + means one or more, ? means optional, and parentheses group expressions.

Concatenation is implicit. For example, ab means a followed by b. The program inserts an internal dot operator only after validation. The user can write # for epsilon.

3. MAIN.JAVA

Main.java contains only the main method. It creates a UI object and calls start(). This keeps the program entry point simple and easy to explain.

4. UI.JAVA

UI.java owns the console menu. Option 1 lets the user enter any regex. Options 2, 3, and 4 display the NFA, DFA, and minimized DFA. Option 5 tests custom strings. Option 6 runs the professor demo test case.

When a regex is entered, UI resets state numbering, calls RegexParser, builds the NFA, converts it to DFA, minimizes it, and stores all results so they can be displayed later.

5. REGEXPARSER.JAVA

RegexParser validates the expression before any automaton is built. It checks empty input, wrong operator positions, missing operands, and unbalanced parentheses.

After validation, it inserts explicit concatenation internally. Example: ab becomes a.b.

Then it uses the shunting-yard algorithm to convert infix regex into postfix regex, because Thompson construction is easier on postfix input.

6. STATE.JAVA AND NFA.JAVA

State.java represents one NFA state. Each state has an id such as q0 and a map of transitions. A transition maps one symbol to a set of target states, because an NFA can have many choices for the same symbol.

NFA.java stores the start state, final states, all states, and alphabet. It also prints a readable list of transitions for presentation.

7. THOMPSONCONSTRUCTION.JAVA

ThompsonConstruction builds small NFA fragments and combines them. For a normal symbol, it creates two states with one transition between them.

For concatenation, final states of the left fragment connect by epsilon to the start of the right fragment. For union, a new start branches by epsilon to both fragments and both fragments connect to a new final state.

For star, plus, and optional, the class adds epsilon transitions according to the formal definitions of those operators.

8. SUBSETCONSTRUCTION.JAVA

SubsetConstruction converts the NFA to a DFA. Each DFA state is a set of NFA states. The first DFA state is epsilon-closure of the NFA start state.

For each DFA state and each alphabet symbol, the program applies move(), then epsilonClosure(). If that set has not appeared before, a new DFA state is created.

9. DFASTATE.JAVA AND DFA.JAVA

DFAState.java stores the set of NFA states represented by one DFA state. It also stores whether the DFA state is final.

DFA.java stores all DFA states, alphabet, start state, and final states. It prints a transition table and marks start states with -> and final states with *.

10. DFAMINIMIZER.JAVA

DFAMinimizer uses partition refinement. It first separates final states from non-final states. Then it repeatedly splits groups whose transitions go to different groups.

When no group can be split again, each group becomes one state in the minimized DFA. A trap state is added internally when needed to complete missing transitions.

11. STRINGTESTER.JAVA

StringTester starts at the minimized DFA start state. For every input character, it follows exactly one transition. If the final state after the whole string is a DFA final state, the string is accepted.

It also prints a trace such as D0 --a--> D1 so the professor can see how the input was processed.

12. PROFESSOR DEMO

Menu option 6 runs this regex: (a|b)*abb. The language is all strings over {a,b} that end with abb.

Expected accepted strings: abb, aabb, bababb. Expected rejected strings: ab, aba, bbb.

After the demo runs, the minimized DFA remains loaded, so the user can choose option 5 and enter any custom test string.

13. HOW TO RUN

Command line: `cd RegexToDFAProject`, then `javac -d out src\*.java`, then `java -cp out Main`.

In IntelliJ: open the `RegexToDFAProject` folder, mark `src` as Sources Root if needed, open `Main.java`, and click Run.

In NetBeans: create a Java project, copy the `src` files into its `src` folder, set `Main` as the main class, and run.